The weekend decision

Outlook	Temp	Humidity	Wind	Play?
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No
Sunny	**Cool**	**High**	**Strong**	**???**

14 past weekends: did we **go out and play** (Yes/No), given the weather?

Today is **Sunny, Cool, High humidity, Strong wind**.

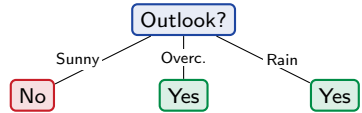
Would you go out?

This is the classic *Play Tennis* dataset (9 Yes, 5 No) – our Armenian "*go to Sevan this weekend?*" We will learn to decide by asking a few yes/no questions.

Predict from one feature

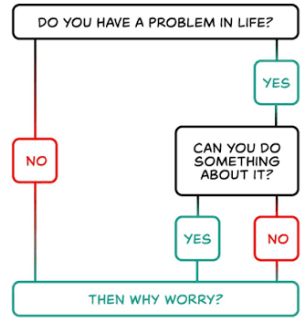
Simplest possible model: look at **one** feature and vote the majority. Take **Outlook**:

Outlook	Yes	No	majority
Sunny	2	3	No
Overcast	4	0	Yes (pure!)
Rain	3	2	Yes



Outline

What a tree is
Why we need impurity
Impurity measures
Growing a tree (CART)
Overfitting and pruning
In practice and bridge



A decision tree you have already used.

What a tree is

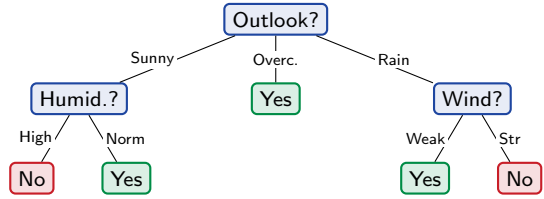
A model you can read: a chain of yes/no questions down to an answer.

Sunny and Rain are still mixed – ask another question

Sunny $(2+, 3-)$ and Rain $(3+, 2-)$ are impure. Split each again – Sunny by **Humidity**, Rain by **Wind**:

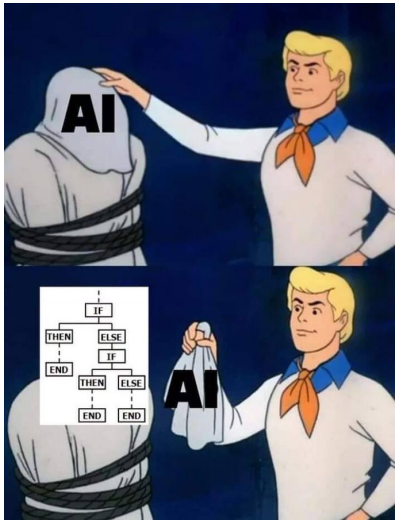
A tree is just **nested if/else** rules:

```
if Outlook = Overcast:  Yes
elif Outlook = Sunny:
  if Humidity = High:  No else:  Yes
elif Outlook = Rain:
  if Wind = Strong:  No else:  Yes
```



The if/else block and the tree are *the same model* – two ways to write it.

A tree is nested if/else – that is all



Strip the mystique: a trained decision tree is just a pile of nested `if/else` rules – questions and answers, no magic.

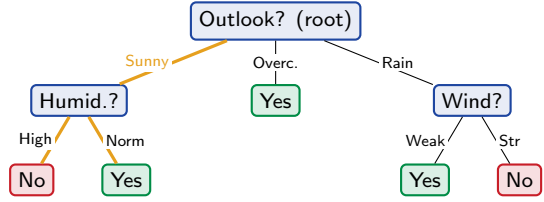
The rest of this chapter is only about *which* question to ask at each node, and in what order.

(meme, unknown origin)

Anatomy of a tree

- **Root**: the first question (top).
- **Internal node**: a question on one feature.
- **Branch**: an answer to that question.
- **Leaf**: a final prediction (no more questions).
- **Depth**: longest root-to-leaf path.

Prediction = walk from the root, follow the branch that matches the row, output the leaf you land in.

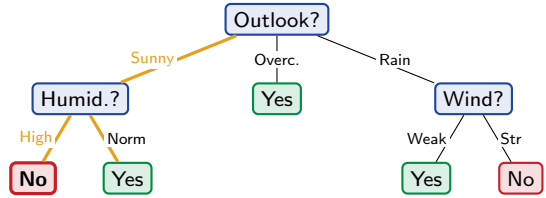


Predicting today (Sunny, Cool, High, Strong)

Walk the tree:

1. Outlook = **Sunny** → go left.
2. Humidity = **High** → leaf.
3. Predict **No**.

That "No" is backed by **3 training rows** (all Sunny + High weekends said No). We did not even need Temperature or Wind.



Why we need impurity

To grow a tree we must score a split: how much purer did the node get?

Feature order matters

We started with Outlook. What if we had started with **Temperature**?

Outlook first (our tree)

- Today → Sunny → High → No.
- Leaf backed by **3 rows**.

Temperature first (another tree)

- Today → Cool → Sunny → . . . only **1 matching row** (D9, which was Normal/Weak) → Yes.

Same data, **different feature order** ⇒ **different tree** ⇒ **different prediction** for today (No vs Yes). Which one should we trust?

Predict-first: which tree is better?

One tree predicts **No** from a leaf resting on **3** weekends; the other predicts **Yes** from a leaf resting on **1** weekend (that also differed in humidity and wind).

Which prediction would you trust?

Predict-first: which tree is better?

One tree predicts **No** from a leaf resting on **3** weekends; the other predicts **Yes** from a leaf resting on **1** weekend (that also differed in humidity and wind).

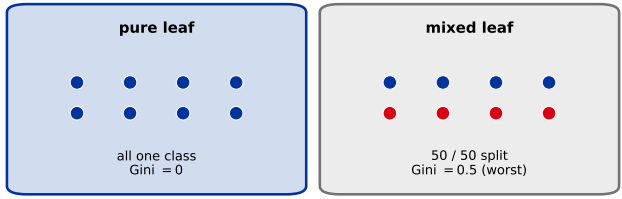
Which prediction would you trust?

The one backed by **more rows**. A leaf covering many consistent examples generalizes; a leaf covering one example is memorizing.

Smaller trees usually generalize better – their leaves rest on more rows (higher coverage). A good split order reaches *pure, well-covered* leaves *sooner*. (We will make this precise as overfitting in Section 5.)

So: split on whatever increases purity the most

We want pure leaves *sooner* $\Rightarrow$ at each node, pick the feature whose split makes the children **as pure as possible**. We need a number for how **impure** (mixed) a leaf is – *zero* when it is all one class, *large* when it is 50/50:



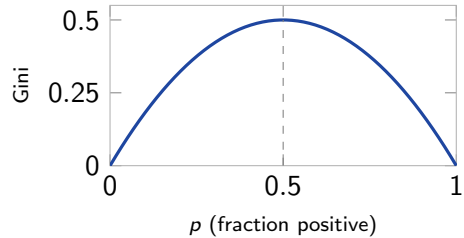
Two standard measures do exactly this: **Gini** and **entropy**.

Impurity measures

Gini and entropy: two ways to put a number on how mixed a node is.

Gini impurity (and its famous cousin)

$$\text{Gini} = 1 - \sum_k p_k^2 \quad \text{(binary: } 2p(1-p)\text{)}.$$



Reading it: draw *two* points at random (with replacement); Gini = P(they are **different** classes).

50/50: P(differ) $= 0.5{\cdot}0.5 + 0.5{\cdot}0.5 = 0.5$

(red-then-blue or blue-then-red) $= 1 - \sum_k p_k^2$.

- pure: the two draws always match $\Rightarrow$ Gini 0.
- 50/50: 0.5 (maximally mixed); 80/20: 0.32.

The economic cousin. Named after Corrado Gini (1912). The economics *Gini index* of income inequality is a sibling measure – both ask *"how different are two random members of a population?"* (here, their class; in economics, their income). Same idea, different formula.

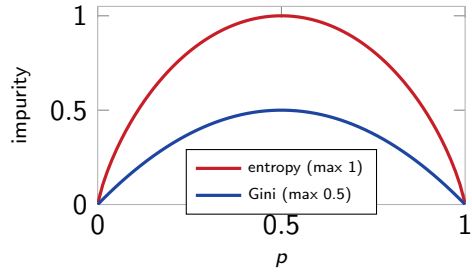
Entropy and information gain

Entropy (in bits) is another impurity measure:

$$H = -p \log_2 p - (1-p) \log_2 (1-p).$$

The split's value is the **information gain** – impurity removed:

$$\mathrm{IG} = H_{\text{parent}} - \sum_{\text{child } c} \frac{n_c}{n} H_c.$$



Same idea for Gini ("Gini gain"). Pick the split with the biggest drop.

Both peak at $p = 0.5$ and vanish at the pure ends – they mostly agree.

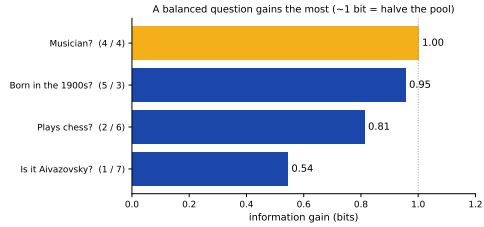
Akinator: 20 questions is information gain in disguise

The web genie **Akinator** guesses your character from yes/no questions – it keeps a **candidate set** and asks whatever **shrinks it fastest**, i.e. maximizes information gain.

Pool of 8 Armenians (Aznavour, Komitas, Petrosyan, Aivazovsky, . . .),
$H_{\text{parent}} = \log_2 8 = 3$ bits:

- **"Musician?"** $4/4$: children 2 bits each, $\mathrm{IG} = \mathbf{1}$ bit.
- **"Is it Aivazovsky?"** $1/7$: $\mathrm{IG} = 3 - 2.46 = \mathbf{0.54}$ bit.

A **balanced** question buys a whole bit; a narrow one barely helps.



A tree splits the *same* way – pick the question that removes the most uncertainty. **20 balanced questions** separate $2^{20} \approx 1{,}000{,}000$ characters: why *informative* splits build *short* trees.

Worked by hand: Gini gain of the Outlook split

Parent node (9 Yes, 5 No):

$$\text{Gini} = 1 - \left(\tfrac{9}{14}\right)^2 - \left(\tfrac{5}{14}\right)^2 = 1 - 0.413 - 0.128 = \mathbf{0.459}.$$

Children after splitting on Outlook:

child	counts	Gini
Sunny	2 Yes, 3 No	$1 - (\frac{2}{5})^2 - (\frac{3}{5})^2 = 0.48$
Overcast	4 Yes, 0 No	$1 - 1 - 0 = 0$ (pure!)
Rain	3 Yes, 2 No	$1 - (\frac{3}{5})^2 - (\frac{2}{5})^2 = 0.48$

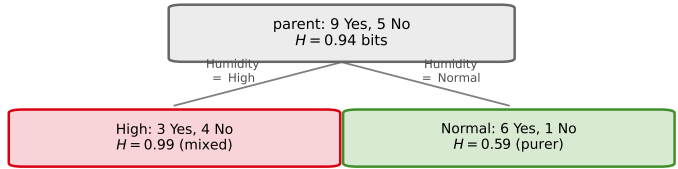
Weighted child impurity (weight = child rows /14 – a leaf covering more rows counts more; this is the *expected* impurity after the split), then the gain:

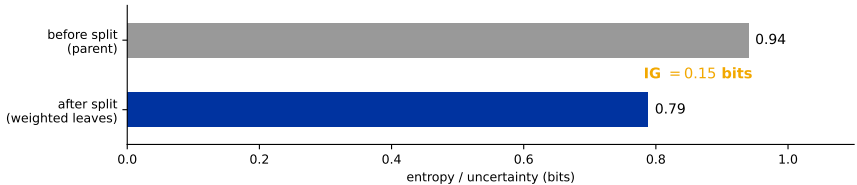
$$\tfrac{5}{14}(0.48) + \tfrac{4}{14}(0) + \tfrac{5}{14}(0.48) = 0.343 \;\Rightarrow\; \text{gain} = 0.459 - 0.343 = \mathbf{0.116}.$$

Overcast is a *free pure leaf* (Gini = 0) – that is why Outlook scores so well. Do this for every feature; keep the biggest gain.

Information gain, visually: uncertainty removed

A split's **information gain** = the drop in entropy from the parent to its (row-weighted) leaves. Splitting the 14 weekends on `Humidity`:





Parent uncertainty 0.94 bits → weighted leaves 0.79 bits, so $\mathrm{IG} = 0.94 - 0.79 = 0.15$. CART scores *every* feature this way and keeps the biggest gain – the next slide does exactly that.

Worked split: which feature is the best root?

Parent entropy (9 Yes, 5 No): $H = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940$ bits. Information gain of each feature:

split on		information gain
Outlook	**0.247**	← winner (Overcast is pure)
Humidity		0.152
Wind		0.048
Temperature		0.029

Outlook removes the most impurity, so CART makes it the root – exactly the tree we drew by hand.

These numbers use **entropy / information gain**. sklearn defaults to **Gini** – it picks the same winner here and gives a near-identical tree. Do not be surprised the criterion silently changes between this slide and the sklearn one.

The trap: information gain loves many-valued features

Predict-first: add a `Day` column (14 distinct dates, one per row) to the weather data. Which feature does information gain pick as the root?

The trap: information gain loves many-valued features

Predict-first: add a `Day` column (14 distinct dates, one per row) to the weather data. Which feature does information gain pick as the root?

Day wins - and it is useless. Splitting on Day gives 14 pure single-row leaves, so its children have zero impurity:

$$\mathrm{IG}(\texttt{Day}) = H_{\text{parent}} - 0 = 0.940 \text{ bits} \;>\; \mathrm{IG}(\texttt{Outlook}) = 0.247.$$

That 14-way split is the **ID3** picture. **sklearn's CART splits only binary**, so it never carves 14 leaves at once – *but* a many-valued feature still offers **many candidate cut-points**, so raw gain stays **biased toward high-cardinality features** (even a useless ID), just milder. Fixes: **gain ratio** (C4.5) normalizes by the split's own entropy; and high-cardinality categoricals need care (CatBoost's target encoding, [20]).

Regression trees: same idea, numeric target

Predict a *number* (apartment rent from area). Split to reduce **spread** (variance / SSE) instead of class impurity; each leaf predicts the **mean** of its rows.

Predict-first: what does the tree predict for an area *between* two training points?

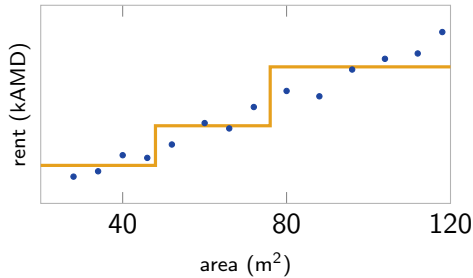
Regression trees: same idea, numeric target

Predict a *number* (apartment rent from area). Split to reduce **spread** (variance / SSE) instead of class impurity; each leaf predicts the **mean** of its rows.

Predict-first: what does the tree predict for an area *between* two training points?

The leaf **mean** – a **flat step**, not a smooth line. A tree's prediction is piecewise-constant.

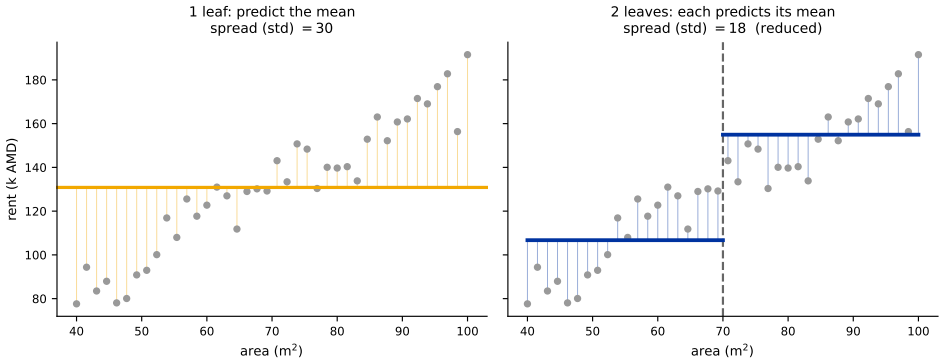
Deeper tree ⇒ more steps ⇒ closer fit (and, eventually, overfitting). Seeds the gradient-boosting residual idea in [19].



Step function: one constant per leaf region.

Regression split: reduce the spread, predict the mean

No Gini for a number – a split is scored by how much it **reduces spread** (variance), and each **leaf predicts the mean** of its rows.



One leaf (left) predicts a single mean for everyone – big residuals. One split (right) makes two leaves, each with its own mean, and the spread (std) drops from **30** to **18**. Keep splitting ⇒ tighter leaves.

Growing a tree (CART)

The greedy, one-split-at-a-time algorithm sklearn actually runs.

The CART algorithm

Grow(node):

1. If pure or a stop rule fires $\rightarrow$ make a leaf (majority class / mean).
2. Else try every (feature, split) and score by impurity drop.
3. Keep the best split; send rows to the children.
4. **Recurse** on each child.

Numeric features can't split "by value" – they need a **threshold** (age > 28?). How that threshold is chosen is the next two slides.

Play Tennis is all-categorical, so the threshold mechanic only shows up on numeric data (e.g. Titanic age / `fare`).

CART = *Classification And Regression Trees*: same recursion, only the impurity (Gini/entropy vs variance) and the leaf output (class vs mean) change.

Choosing a split on a numeric feature

A numeric feature needs a **threshold**. The exact recipe:

1. Sort the rows by the feature.
2. Candidate thresholds = **midpoints** between adjacent values.
3. Score each by impurity drop; keep the best.

Example – feature age, label survived (1) / died (0), sorted:

age	20	25	30	40	50
y	0	0	1	1	1

candidate midpoints: 22.5, **27.5**, 35, 45.

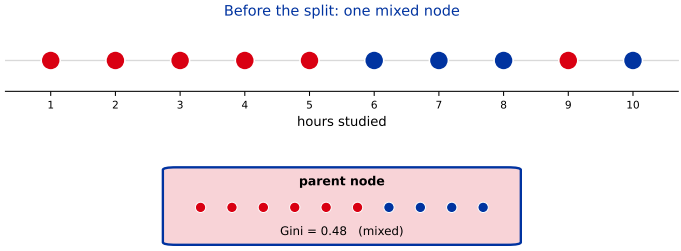
Gini gain (parent Gini = 0.48):

threshold	gain
22.5	0.18
27.5	**0.48**
35	0.21
45	0.08

`age > 27.5` splits it perfectly (two pure leaves). *Shortcut:* only thresholds where the *label changes* can ever win, so you skip the rest.

Watch one split get chosen

Ten students, one feature (`hours studied`); the whole node is **mixed**. A split sends points **left / right** into two child leaves – which threshold makes those leaves **purest**?

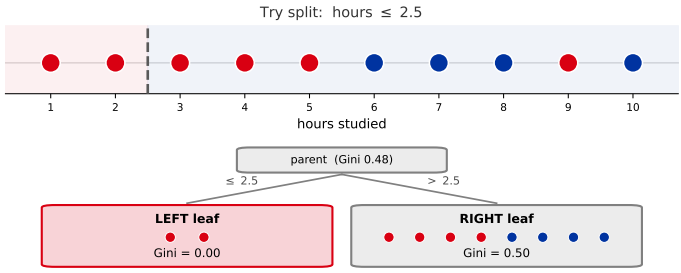


Scorecard (gain = parent Gini - weighted leaves)
parent Gini = 0.48

failed passed

Watch one split get chosen

Each threshold splits the node into a **left leaf** and a **right leaf**. **Gini gain** = parent − the row-weighted average of the two leaf Ginis.

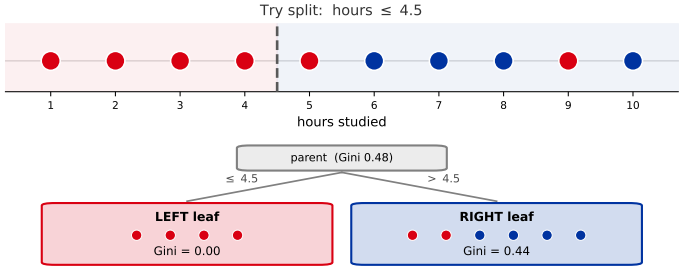


Scorecard (gain = parent Gini - weighted leaves)
parent Gini = 0.48
hours ≤ 2.5 : weighted Gini 0.40 gain = 0.08

failed passed

Watch one split get chosen

Each threshold splits the node into a **left leaf** and a **right leaf**. **Gini gain** = parent − the row-weighted average of the two leaf Ginis.

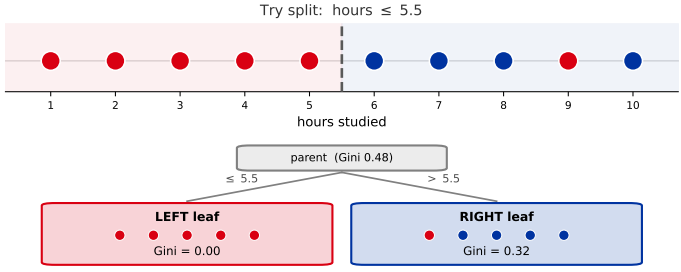


Scorecard (gain = parent Gini - weighted leaves)
parent Gini = 0.48
hours ≤ 2.5 : weighted Gini 0.40 gain = 0.08
hours ≤ 4.5 : weighted Gini 0.27 gain = 0.21

● failed ● passed

Watch one split get chosen

Each threshold splits the node into a **left leaf** and a **right leaf**. **Gini gain** = parent − the row-weighted average of the two leaf Ginis.



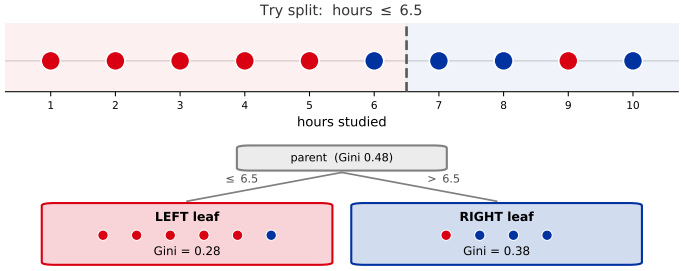
Scorecard (gain = parent Gini - weighted leaves)
parent Gini = 0.48

hours ≤ 2.5 : weighted Gini 0.40 gain = 0.08
hours ≤ 4.5 : weighted Gini 0.27 gain = 0.21
hours ≤ 5.5 : weighted Gini 0.16 gain = 0.32

● failed ● passed

Watch one split get chosen

Each threshold splits the node into a **left leaf** and a **right leaf**. **Gini gain** = parent − the row-weighted average of the two leaf Ginis.



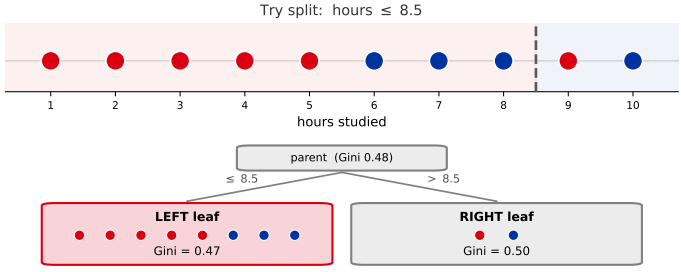
Scorecard (gain = parent Gini - weighted leaves)
parent Gini = 0.48

hours ≤ 2.5 : weighted Gini 0.40 gain = 0.08
hours ≤ 4.5 : weighted Gini 0.27 gain = 0.21
hours ≤ 5.5 : weighted Gini 0.16 gain = 0.32
hours ≤ 6.5 : weighted Gini 0.32 gain = 0.16

● failed ● passed

Watch one split get chosen

Each threshold splits the node into a **left leaf** and a **right leaf**. **Gini gain** = parent − the row-weighted average of the two leaf Ginis.



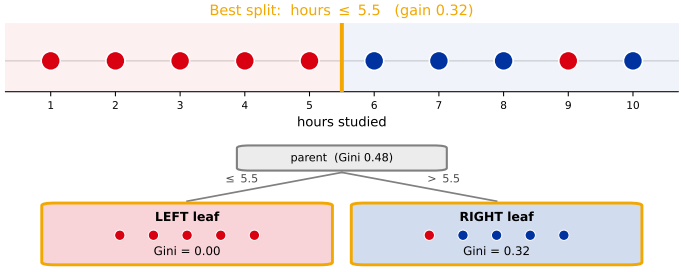
Scorecard (gain = parent Gini - weighted leaves)
parent Gini = 0.48

hours ≤ 2.5 : weighted Gini 0.40 gain = 0.08
hours ≤ 4.5 : weighted Gini 0.27 gain = 0.21
hours ≤ 5.5 : weighted Gini 0.16 gain = 0.32
hours ≤ 6.5 : weighted Gini 0.32 gain = 0.16
hours ≤ 8.5 : weighted Gini 0.47 gain = 0.01

● failed ● passed

Watch one split get chosen

hours ≤ 5.5 **wins**: a *pure* left leaf (Gini 0) and a mostly-pass right leaf give the biggest gain. CART keeps this split, then **recurses** on each leaf.



Scorecard (gain = parent Gini - weighted leaves)
parent Gini = 0.48

hours ≤ 2.5 : weighted Gini 0.40 gain = 0.08
hours ≤ 4.5 : weighted Gini 0.27 gain = 0.21
hours ≤ 5.5 : weighted Gini 0.16 gain = 0.32 <- best
hours ≤ 6.5 : weighted Gini 0.32 gain = 0.16
hours ≤ 8.5 : weighted Gini 0.47 gain = 0.01

● failed ● passed

Scaling up: quantile and histogram splits

Exact search sorts every feature and tests $\sim n$ thresholds *at every node* – fine for thousands of rows, painful for millions. Two faster approximations:

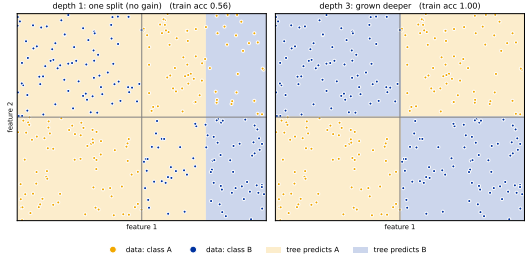
- **Quantile / percentile splits.** Instead of all midpoints, only test thresholds at the feature's quantiles (e.g. the 32 or 256 percentile cut-points). Far fewer candidates, near-identical splits. (XGBoost's "approx" / weighted quantile sketch.)
- **Histogram-based splits.** *Bin* each feature into a fixed number of bins (often 255) **once**; a split then only scans bin edges and sums per-bin counts. Cost drops from $O(n)$ to $O(\#\text{bins})$, and the bins are reused at every node. This is how **LightGBM**, `HistGradientBoosting`, and XGBoost (`hist`) get their speed (callback: the LightGBM frame).

Same idea as exact search – just *fewer* candidate thresholds. A tiny accuracy cost (coarser cut-points) buys a big speed-up on large data; for small data, exact is fine.

Greedy, not optimal

CART is **greedy**: at each node it takes the split that looks best *right now* and never reconsiders.

- Finding the *smallest* tree that fits is NP-hard – we cannot search all trees.
- "Increase purity most" is a cheap **heuristic** for "end up small".
- It can **miss a split that only pays off later** – like XOR (right): one split gains *nothing*, so a greedy "stop when no split helps" rule would quit.



Depth 1 gains nothing ($\approx$ coin flip); grow deeper and CART does separate XOR. The useless-looking first split is the real greedy caveat.

Overfitting and pruning

A tree left unchecked memorizes the training set; pruning buys back generalization.

Predict-first: does an unrestricted tree overfit?

Grow with **no depth limit**. What happens to train vs test accuracy?

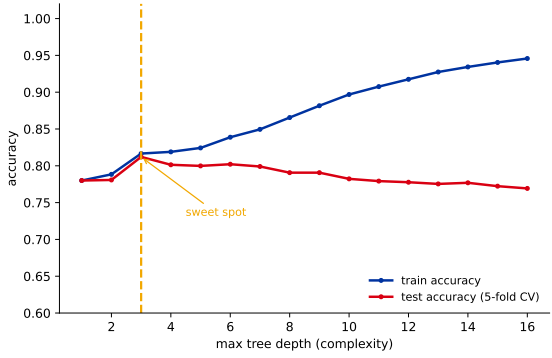
Predict-first: does an unrestricted tree overfit?

Grow with **no depth limit**. What happens to train vs test accuracy?

On Titanic the unrestricted tree grows to **depth 24, 316 leaves**:

- train accuracy **96.5%** – near-perfect, it memorizes.
- test (5-fold CV) only **76%** – *worse* than the depth-1 "`gender`" stump (78%).

A 20-point train–test gap: textbook overfitting.



Depth is the complexity knob (callback: cross-validation). Train keeps rising; test peaks early, then falls.

Only unrestricted depth overfits. A *shallow* tree (depth 2–3, ≈80% here) is the healthy baseline – the depth-2 Titanic tree we draw later is *not* overfit. Depth is a dial: too deep memorizes, too shallow underfits.

Pre-pruning: stop growing early

Set limits *before/while* growing – refuse splits that go too far:

sklearn parameter	**stops a split when...**
`max_depth`	the tree is already this deep
`min_samples_split`	the node has too few rows to split
`min_samples_leaf`	a child would be too small
`max_leaf_nodes`	the tree already has enough leaves
`min_impurity_decrease`	the split barely helps

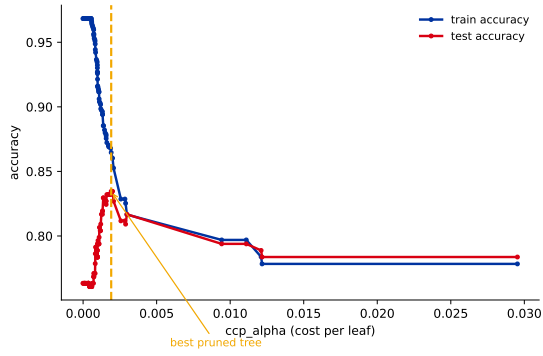
Cheap, needs no extra data. Risk: too aggressive $\Rightarrow$ underfit; it may stop before a good *later* split.

Post-pruning: grow full, then cut back

Grow a big tree, then **remove** subtrees that do not earn their keep.

- **Reduced-error pruning**: cut a subtree if it does not help on a hold-out set.
- **Cost-complexity pruning** (`ccp_alpha`): minimize error $+ \alpha$ (#leaves). Larger $\alpha \Rightarrow$ smaller tree.

On Titanic, pruning lifts **test accuracy to 83.5%** at the best `ccp_alpha` – selective cuts beat a uniform depth cap.



Test peaks at the right α; over-pruning then underfits.

Pre- vs post-pruning, and how to choose

Pre-pruning

- Fast; no extra data.
- Can stop too early (misses good later splits).

Post-pruning

- Slower; needs data.
- Can undo bad greedy splits.

Either way, pick the knob by cross-validation. Sweep `max_depth` or `ccp_alpha`, keep the value with the best CV score. The depth U-curve is the same picture as the cross-validation lecture's complexity-knob plot.

Key hyperparameters and their impact

Almost every tree knob controls **one thing – complexity**. Here they are together, with which way each pushes the over/underfit dial. **Tune by CV**.

hyperparameter	what it controls	raising it ⇒
`max_depth`	longest question chain	more complex (overfit)
`min_samples_leaf`	smallest allowed leaf	simpler (regularizes)
`min_samples_split`	rows needed to split	simpler
`max_leaf_nodes`	total # of leaves	more complex
`min_impurity_decrease`	min gain to allow a split	simpler
`ccp_alpha`	post-pruning strength	simpler (more pruned)
`criterion`	`gini` / `entropy`	≈ no accuracy effect (gini faster)
`max_features`	features tried per split	< all adds randomness (mainly RF, [18])

In practice: tune `max_depth` (or `ccp_alpha`) and `min_samples_leaf` by CV – those two do most of the work. `criterion` barely matters; `class_weight="balanced"` handles imbalance (a separate concern, not complexity).

In practice and bridge

Using trees in sklearn - and why one tree is never the end of the story.

Decision trees in sklearn (the default tool)

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

tree = DecisionTreeClassifier(max_depth=3, ccp_alpha=0.0,
                              random_state=509)
tree.fit(X_train, y_train)        # Titanic features
plot_tree(tree, feature_names=cols, filled=True)
```

On Titanic the tree usually asks gender first, then an age threshold – readable, you can show it to a stakeholder.

No scaling needed – trees only compare thresholds, so monotone rescaling changes nothing (callback: preprocessing). *But* sklearn trees still need **numeric input**: encode categoricals yourself (they are *not* auto-handled, despite folklore).

CART speaks binary: what one-hot does to the tree

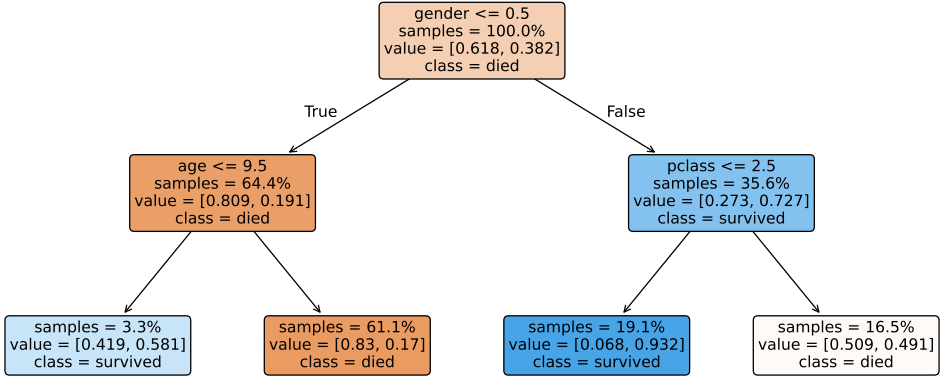
The hand-built tree split `Outlook` **three ways** at once (Sunny / Overcast / Rain) - that is ID3. sklearn's CART only ever asks **one yes/no question** per node.

So you one-hot encode first, and CART recovers the *same decisions* as a chain of binary splits:

$$\texttt{Outlook\_Overcast} \leq 0.5\,? \quad \rightarrow \quad \texttt{Outlook\_Sunny} \leq 0.5\,? \quad \rightarrow \quad \ldots$$

Same tree, different shape. Do not be surprised when the tree you *run* looks deeper and asks `Outlook_Overcast <= 0.5` instead of a tidy 3-way Outlook node - the splits and predictions match, only the drawing differs.

Titanic: a real (depth-2) tree



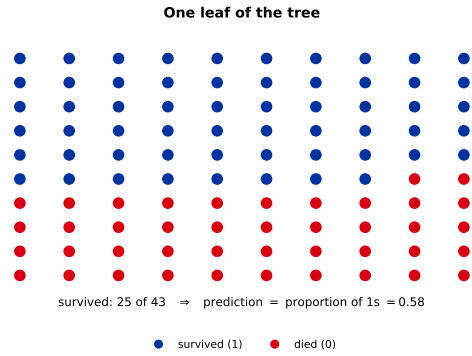
It asks `gender` first (the strongest split), then `age` / `pclass`. Each box shows the split, the % of samples, the class mix, and the colour (orange = died, blue = survived). This depth-2 tree already scores ≈79% – a solid, *non-overfit* baseline you can hand to a stakeholder.

How a leaf predicts: the proportion of 1s

To predict, walk to a leaf and read its training rows:

- **Probability** = fraction of class 1 in the leaf.
- **Class** = the majority (probability > 0.5).

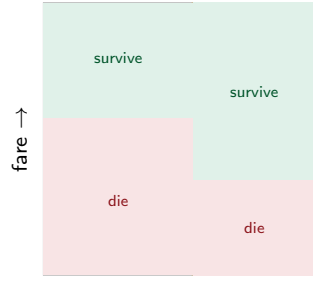
The leaf on the right holds **43** passengers; **25** survived, so the tree predicts $P(\text{survived}) = 25/43 = \mathbf{0.58}$ for anyone who lands there – a genuinely *mixed* leaf, a proportion, not a certainty.



The decision boundary is axis-aligned boxes

Every split is "feature $>$ threshold", so a tree carves the feature space into **axis-aligned rectangles** – one box per leaf, one constant prediction inside.

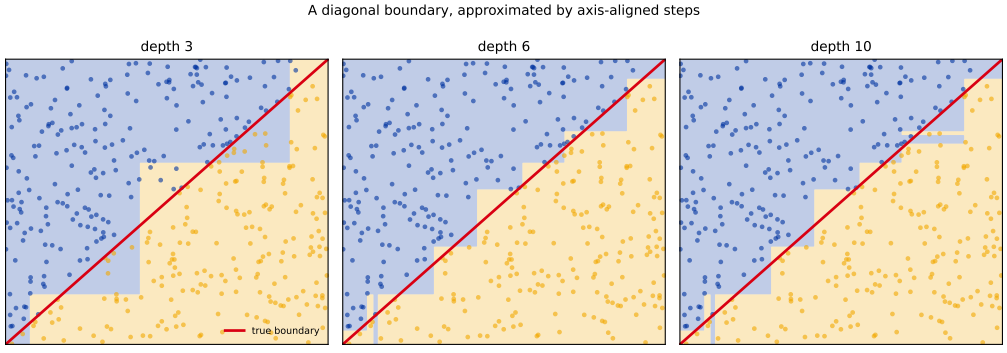
The tree on the right and the boxes are the **same model**: walking the tree = finding which box a point falls in.



Each leaf = one rectangle.

Axis-aligned means staircases

A tree can only cut **straight, axis-aligned** lines. A slanted boundary is approximated by a **staircase** – finer (and more overfit) as depth grows.

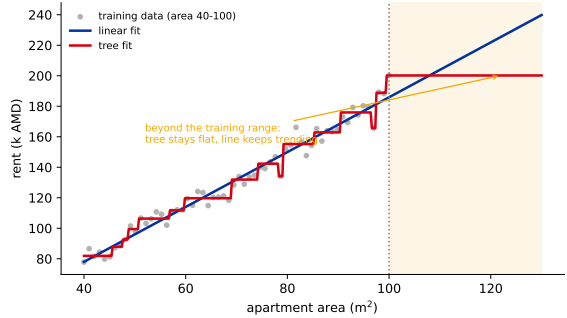


The diagonal (red) is trivial for a line but awkward for a tree – deeper trees chase it with more, smaller boxes. A hint of why we sometimes reach for other models.

Trees cannot extrapolate

A leaf predicts the **mean of its training rows** – a constant. Outside the training range there is no new leaf, so the prediction **flatlines**.

A linear model keeps trending; a tree *and every tree ensemble* does not. It matters whenever you must predict *beyond* what you have seen (prices, demand, time).



Strengths and weaknesses

Strengths

- Interpretable – read the rules.
- No scaling; mixed feature types.
- Captures interactions & non-linearities.
- Automatic feature selection; fast.
- Missing values handled natively (no imputation; sklearn ≥ 1.3).

Weaknesses

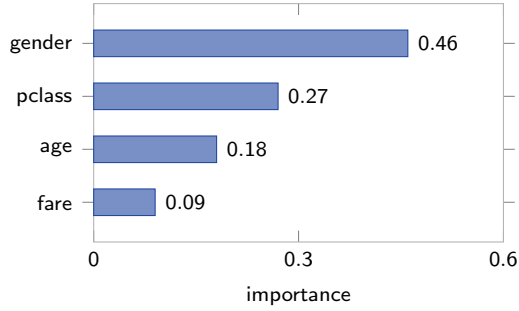
- **High variance / unstable**: change a few rows $\rightarrow$ a very different tree.
- Greedy, not optimal.
- **Axis-aligned only**: a diagonal boundary needs a staircase of splits.
- **Never smooth**; extrapolates poorly (flat beyond the data).

Instability example: on data like the Wisconsin breast-cancer set, dropping a *single* row can flip many predictions and produce a visibly different tree. This variance is the headline weakness – and the reason ensembles exist.

Feature importance (read with care)

A tree scores each feature by **how much impurity its splits removed** (summed over the tree). Quick way to see what drove the model.

Caveat: this impurity-based importance is **biased toward high-cardinality** features (many split points). Prefer *permutation importance*; SHAP comes later (interpretability).



Titanic: `gender` dominates.

The tree is the atom of this chapter

A single tree is weak (high variance). Everything next is built from it.

```
# a single tree, two ways:
DecisionTreeRegressor(max_depth=3)                  # sklearn
LGBMRegressor(n_estimators=1, learning_rate=1.0)  # LightGBM
```

Why `learning_rate=1.0`? LightGBM is a *boosting library*; with the default 0.1 it would shrink the single tree. It also grows *leaf-wise* (knob `num_leaves`) with histogram bins, so it is not byte-identical to sklearn's CART – same idea, different growth.

1 tree → **bag many** = Random Forest (**[18]**) → **boost** them = Gradient Boosting / XGBoost / LightGBM (**[19]–[20]**).

Recap

- A decision tree = **nested yes/no questions**; a leaf gives the prediction (majority class or mean).
- Grow greedily (**CART**): at each node pick the split that drops **impurity** most (**Gini**/entropy for classification, variance for regression).
- Unlimited depth $\Rightarrow$ 0 training error but **overfit**; control with **pre-pruning** and **post-pruning** (`ccp_alpha`), chosen by **CV**.
- Strengths: interpretable, no scaling. Weakness: **high variance**.

Workflow: encode categoricals $\rightarrow$ fit a shallow tree as an interpretable baseline $\rightarrow$ tune `max_depth`/`ccp_alpha` by CV $\rightarrow$ read the rules / importances $\rightarrow$ if you need accuracy over interpretability, reach for ensembles ([18]–[20]).